Thread identifiers are of type `std::thread::id` and can be retrieved in two ways. First, the identifier for a thread can be obtained from its associated `std::thread` object by calling the `get_id()` member function. If the `std::thread` object doesn't have an associated thread of execution, the call to `get_id()` returns a default-constructed `std::thread::id` object, which indicates "not any thread." Alternatively, the identifier for the current thread can be obtained by calling `std::this_thread::get_id()`, which is also defined in the `<thread>` header.

Objects of type `std::thread::id` can be freely copied and compared; they wouldn't be of much use as identifiers otherwise. If two objects of type `std::thread::id` are equal, they represent the same thread, or both are holding the "not any thread" value. If two objects aren't equal, they represent different threads, or one represents a thread and the other is holding the "not any thread" value.

The Thread Library doesn't limit you to checking whether thread identifiers are the same or not; objects of type `std::thread::id` offer the complete set of comparison operators, which provide a total ordering for all distinct values. This allows them to be used as keys in associative containers, or sorted, or compared in any other way that you as a programmer may see fit. The comparison operators provide a total order for all non-equal values of `std::thread::id`, so they behave as you'd intuitively expect: if `a<b` and `b<c`, then `a<c`, and so forth. The Standard Library also provides `std::hash<std::thread::id>` so that values of type `std::thread::id` can be used as keys in the new unordered associative containers too.

Instances of `std::thread::id` are often used to check whether a thread needs to perform some operation. For example, if threads are used to divide work as in listing 2.8, the initial thread that launched the others might need to perform its work slightly differently in the middle of the algorithm. In this case it could store the result of `std::this_thread::get_id()` before launching the other threads, and then the core part of the algorithm